Unsupervised Anomaly Detection in Noisy Surveillance Videos

Atharv Gupta
Department of Electronics and Communication Engineering
Indian Institute of Technology Roorkee
PIXEL PLAY
atharv_g@ee.iitr.ac.in

Abstract

Video anomaly detection is a challenging problem because abnormal events are rare, diverse, and often difficult to define explicitly. In surveillance videos, identifying anomalies requires understanding both spatial appearance and temporal motion patterns. In this work, I explore unsupervised video anomaly detection on the CUHK Avenue dataset as part of the Pixel Play'26 competition. The competition version of the dataset introduces additional challenges due to visual corruption and inconsistencies in the test data.To address these issues, I first design a robust preprocessing pipeline that corrects dataset corruption through bilateral filtering and automatic frame orientation correction. I then explore a series of reconstruction-based anomaly detection models.Starting from a standard convolutional autoencoder baseline, I progressively incorporate temporal modeling using a CNN–ConvLSTM architecture and enhance motion sensitivity through frame-difference modeling.

1 Introduction

Video anomaly detection aims to identify events in surveillance footage that deviate from normal patterns of activity. In many real-world surveillance settings, anomalous events are not known or given to us(ANOMALY!), and only normal video data is available during training. As a result, anomaly detection is naturally formulated as an unsupervised learning problem, where models are required to learn regular spatio-temporal behavior from the training set and detect deviations from it at the testing time.In this work, I focus on frame-level anomaly detection in camera surveillance videos. The task is challenging due to visual corruption in the data and subtle temporal irregularities that are often difficult to distinguish from normal motion. These challenges require methods that are robust to noise while remaining sensitive to deviations in motion patterns.To address this, I adopt a pipeline that combines preprocessing, motion-aware representation learning, and temporal smoothing of anomaly scores during testing.

2 Dataset Analysis

The dataset consists of surveillance videos of a university recorded from a static camera. All training videos contain only normal behavior, while test videos may include anomalies.

2.1 Observed Anomaly Types

The types of Anomalies include:

- motion in the wrong direction,

- running people,
- People throwing objects and loitering.

Figure 1: Some types (NOT ALL) of Anomalies present in Dataset

—

3 Data Corruption and Preprocessing

3.1 High-Frequency Visual Noise

The dataset has significant high-frequency noise resembling TV static. This noise severely degrades frame reconstruction quality and corrupts frame-difference signals.

3.1.1 Bilateral Filtering

To mitigate this issue, bilateral filtering was applied to each frame. Unlike Gaussian smoothing, bilateral filtering preserves edges while suppressing noise, making it suitable for motion analysis.

Figure 2: Effect of bilateral filtering on noisy surveillance frames

3.2 Vertical Flip Correction

Several test videos were found to be vertically flipped. This was corrected using a statistical property of the scene: the lower half of the frame is consistently brighter than the upper half in normal orientation. Frames violating this condition were flipped vertically.

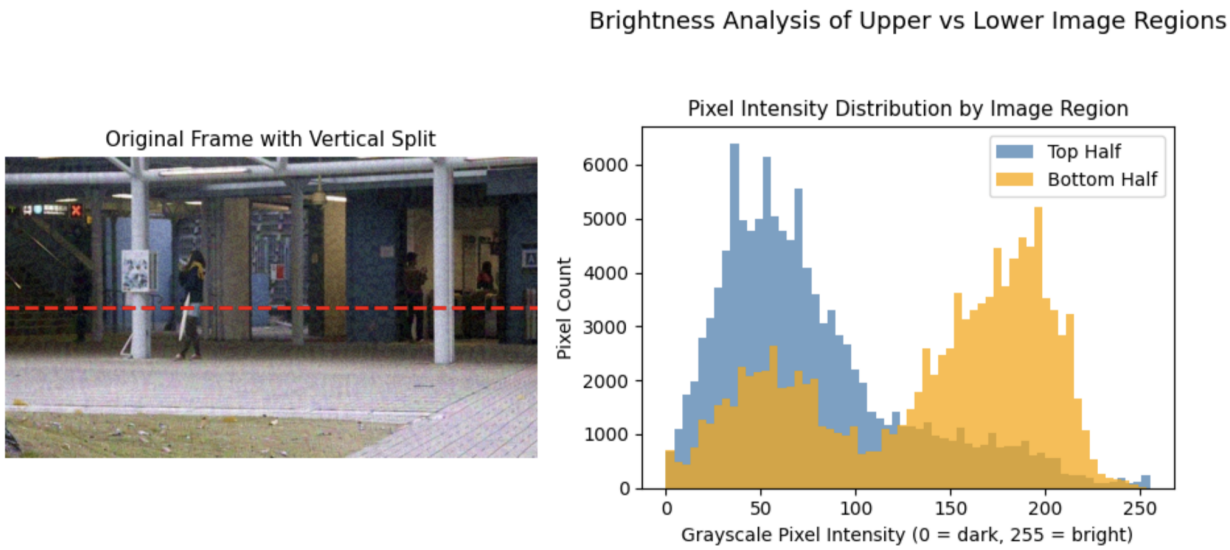


Figure 3: Statistics of the above figure: Top half mean intensity: 76.62, Bottom half mean intensity: 136.34

3.3 Gray scaling Input

It is quite clear that most of the anomaly in the dataset is based on MOTION rather than being color/appearance dependent. Therefore,changing the image into grayscale felt like the obvious next step. Moreover it also has the following advantages

- Computationally less complex,
- Helps to remove noise from the data as the TV-Static is color based,
- People throwing objects and loitering.

4 Model Architectures

4.1 2D convolutional autoencoder

As a first baseline, I used a convolutional autoencoder to model normal visual patterns in surveillance frames. The motivation behind this choice is straightforward: when trained only on normal data, an autoencoder learns to reconstruct regular scenes well, while unusual or unseen events tend to produce higher reconstruction error.

4.1.1 Architecture Overview

The model follows a symmetric encoder-decoder structure. The encoder progressively compresses the input frame into a low-dimensional latent representation using convolutional layers, while the decoder attempts to reconstruct the original frame from this compressed feature space.

4.1.2 Encoder

The encoder consists of a stack of convolutional layers with increasing channel depth. Each convolution is followed by batch normalization and a ReLU non-linearity to stabilize training and improve feature learning.

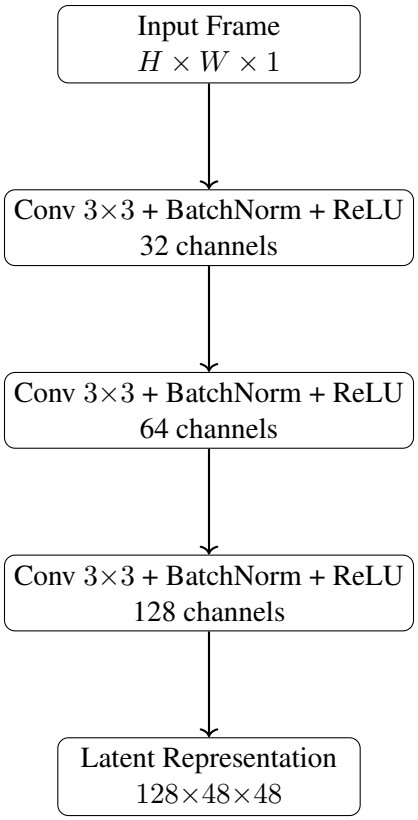


Figure 4: Encoder architecture of the baseline convolutional autoencoder.

After the final encoder layer, the spatial resolution is reduced while information about normal appearance is preserved. This compressed representation forms the latent space of the autoencoder.

4.1.3 Latent Representation

The latent representation captures the dominant spatial patterns present in normal training frames. Since the model is trained exclusively on normal data, this representation is biased toward reconstructing typical background motion and appearance. Abnormal events, therefore are poorly reconstructed.

4.1.4 Decoder

The decoder mirrors the encoder structure and gradually upsamples the latent representation back to the original spatial resolution.

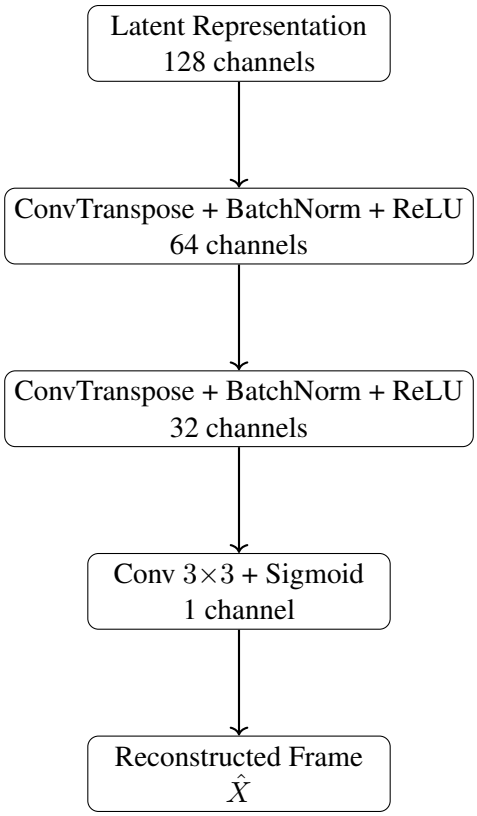


Figure 5: Decoder architecture of the baseline convolutional autoencoder.

The final output is a reconstructed grayscale frame $\hat{X}$ with the same spatial dimensions as the input.

4.1.5 Training Objective

The model is trained using a pixel-wise reconstruction loss. Given an input frame X and its reconstruction $\hat{X}$, the loss is defined as:

$$\mathcal{L}_{rec} = \frac{1}{HW} \sum_{i=1}^{H} \sum_{j=1}^{W} (X_{ij} - \hat{X}_{ij})^2$$

4.1.6 Anomaly Scoring

During inference, the reconstruction error for each frame is computed using the same mean squared error loss. Frames with higher reconstruction error are therefore assigned higher anomaly scores, indicating a greater likelihood of abnormal behavior.

4.1.7 Limitations of the Baseline Model

Since a 2D convolutional autoencoder processes each frame independently,it lacks explicit temporal(time) modeling. As a result, the model has no understanding of motion or how to relate motion across frames. Many anomalies in the dataset are defined primarily by how motion evolves over time (e.g., running, loitering, or moving in the wrong direction). Since the autoencoder only models spatial appearance, it often fails

to strongly penalize such temporal irregularities. These limitations motivate the use of spatio-temporal models that explicitly capture motion patterns, such as CNN–ConvLSTM architectures and frame-difference-based representations, which are explored in subsequent sections.

4.1.8 Mean Average Precision

The performance of all models is evaluated using Mean Average Precision (mAP) on 29% of the testing data,Unlike accuracy, mAP is more suitable for highly imbalanced settings, where anomalous frames are rare compared to normal frame. A higher mAP indicates that anomalous frames consistently receive higher scores than normal frames across the dataset.

4.1.9 Result

The model achieved a mean average precision (mAP) score of:

$$\text{mAP} = \mathbf{0.45375}$$

4.2 CNN + ConvLSTM

The CNN-ConvLSTM model works by observing a short sequence of consecutive frames and learning to predict what the next frame should look like under normal behavior. During training, the model is trained only on normal video sequences and therefore learns typical motion patterns present in the scene .At inference time, a sequence of T frames is provided as input, and the model predicts the next frame in the sequence. If the real next frame matches the predicted one closely, the reconstruction error remains low. However, when abnormal motion occurs, such as unexpected direction changes or irregular behavior, the model fails to predict the next frame accurately, resulting in a higher reconstruction error.

4.2.1 CNN Encoder

The encoder is responsible for extracting spatial features from individual video frames. Each input frame is first converted to grayscale and passed through a stack of convolutional layers. These layers capture low-level visual patterns such as edges and textures, as well as higher-level spatial structures relevant to normal scene appearance. Each convolution layer is followed by batch normalization and a ReLU activation to stabilize training and make it faster. The encoder processes each frame independently but shares weights across time steps.

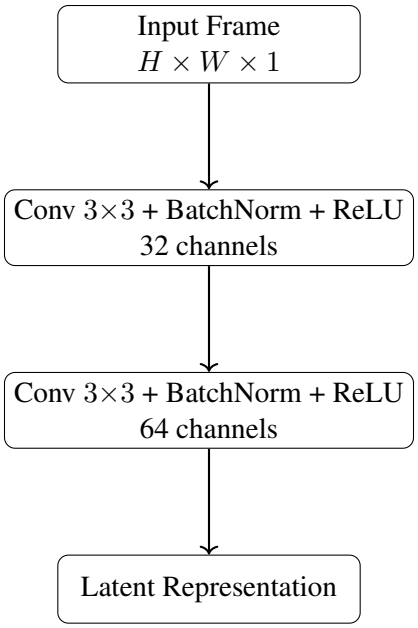


Figure 6: Encoder architecture

4.2.2 ConvLSTM

In order for the model to understand the temporal dependencies and motion, I use a convolutional LSTM (ConvLSTM).Unlike standard LSTM, which applies fully connected operations, ConvLSTM replaces all matrix multiplications with convolutions.Given a sequence of encoder outputs $\{X_t\}_{t=1}^{T}$, where $X_t \in \mathbb{R}^{C\times H\times W}$, the ConvLSTM processes one feature map at a time. At each time step, the current input X_t is concatenated with the previous hidden state H_{t-1} and passed through a convolutional layer that jointly computes the input, forget, output, and candidate gates.These gates control how much information from the previous memory is retained, how much new motion information is written, and what information is exposed to the next layer. All operations are convolutional, allowing the model to preserve spatial structure while modeling temporal dependencies. After processing the full input sequence, the final hidden state H_T summarizes spatio-temporal dynamics across the observed frames and is used by the decoder to predict the next frame in the sequence.

4.2.3 ConvLSTM Update Equations

The encoded feature map X_t is concatenated with the previous hidden state H_{t-1} along the channel dimension and passed through a convolution layer:

$$Z_t = \text{Conv}\left([X_t, H_{t-1}]\right) \tag{1}$$

The output Z_t is split into four components corresponding to the input gate, forget gate, output gate, and candidate memory:

$$Z_t = [I_t, F_t, O_t, G_t] \tag{2}$$

Gate activations are computed as:

$$i_t = \sigma(I_t) \tag{3}$$
$$f_t = \sigma(F_t) \tag{4}$$
$$o_t = \sigma(O_t) \tag{5}$$
$$g_t = \tanh(G_t) \tag{6}$$

The cell state is updated by selectively forgetting past information and incorporating new candidate memory:

$$C_t = f_t \odot C_{t-1} + i_t \odot g_t \tag{7}$$

The hidden state, which represents the output of the ConvLSTM at time step t, is computed as:

$$H_t = o_t \odot \tanh(C_t) \tag{8}$$

4.2.4 ConvLSTM Cell Structure

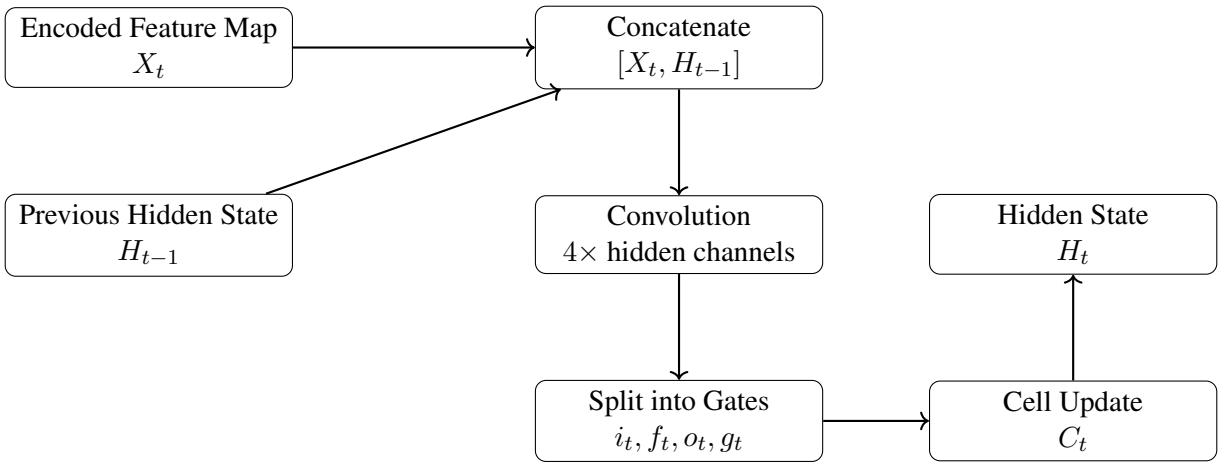


Figure 7: Internal structure of a ConvLSTM cell used for temporal modeling.

4.2.5 Temporal Processing

Given a sequence of T input frames $\{X_{t-T+1}, \ldots, X_t\}$, the CNN encoder extracts spatial features from each frame independently using shared weights. These features are then processed sequentially by the ConvLSTM, which updates its hidden and cell states at each time step. The final hidden state summarizes the spatio-temporal dynamics of the input sequence and is passed to the decoder for next-frame prediction.

This design enables the model to learn normal motion patterns over time, making it suitable for detecting anomalies that arise from irregular temporal behavior.

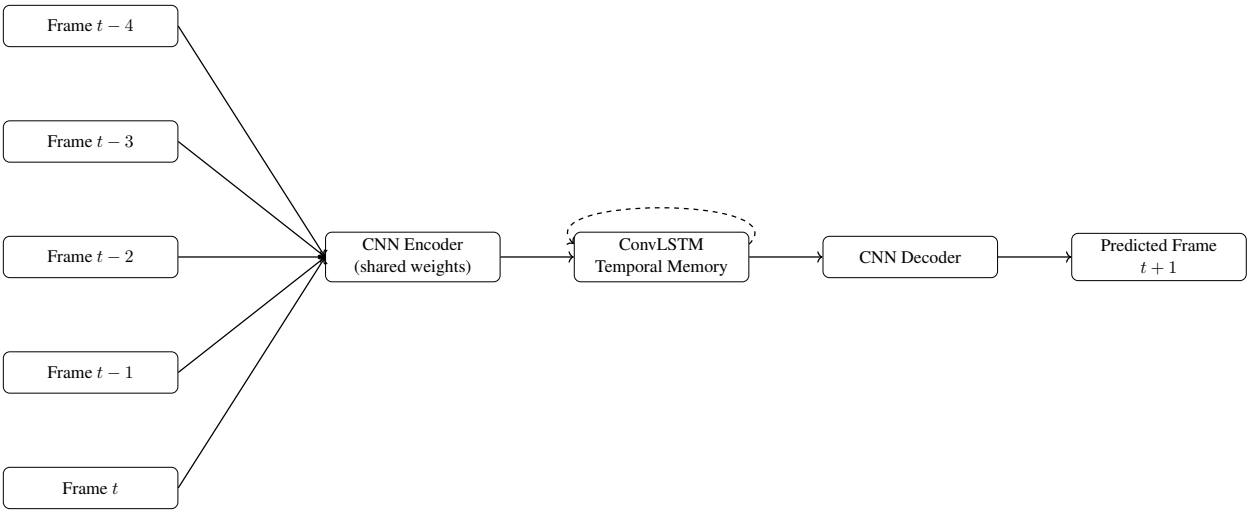


Figure 8: CNN + ConvLSTM architecture for next-frame prediction.

4.2.6 Decoder

The decoder reconstructs the next video frame from the final hidden state of the ConvLSTM. It mirrors the encoder architecture using upsampling and convolution layers to gradually restore spatial resolution. The output is a single-channel grayscale frame of size 192×192, representing the predicted next frame

4.2.7 Training Objective

The model is trained to predict the next frame in a sequence given the previous T frames. Let X_{t+1} denote the ground-truth next frame and $\hat{X}_{t+1}$ the predicted frame. The training objective minimizes the pixel-wise reconstruction error using mean squared error (MSE):

$$\mathcal{L} = \frac{1}{HW} \sum_{i=1}^{H} \sum_{j=1}^{W} \left(\hat{X}_{t+1}^{(i,j)} - X_{t+1}^{(i,j)} \right)^2 \tag{9}$$

A higher reconstruction error at inference time indicates a higher likelihood of anomalous behavior.

4.2.8 Optimization Details

The network is trained using the Adam optimizer with a learning rate of 10^{-4}. The ConvLSTM hidden channel size is set to 96. Training is performed with a batch size of 8.

4.2.9 Anomaly Scoring

During inference, the trained model is applied in a sliding-window manner over each test video. For a given time step t, the model receives a sequence of $T = 5$ consecutive frames $\{X_{t-4}, \ldots, X_t\}$ and predicts the next frame $\hat{X}_{t+1}$.

An anomaly score is computed using the reconstruction error between the predicted frame and the actual next frame X_{t+1}. Specifically, the frame-level anomaly score s_{t+1} is defined as the mean squared error:

$$s_{t+1} = \frac{1}{HW} \sum_{i=1}^{H} \sum_{j=1}^{W} \left(\hat{X}_{t+1}^{(i,j)} - X_{t+1}^{(i,j)} \right)^2 \tag{10}$$

Frames that follow regular motion patterns are reconstructed accurately and yield low anomaly scores, while frames containing abnormal motion or unexpected events result in higher prediction errors.

4.2.10 Result

The model achieved a mean average precision (mAP) score of:

$$\textbf{mAP} = 0.602$$

This result indicates a significant improvement over the convolutional autoencoder baseline. This performance gain highlights the importance of temporal modeling for detecting motion-based anomalies

4.3 Frame-Difference Enhanced CNN + ConvLSTM

While the CNN + ConvLSTM model captures temporal dynamics through LSTM's, it still relies on the encoder to implicitly learn motion information from raw frames. However, in our dataset, anomalies are often characterized by abrupt or irregular motion, which can be made more explicit through frame differencing. To strengthen motion sensitivity, frame differences are introduced as an additional input stream. Given a sequence of grayscale frames $\{X_{t-T+1}, \ldots, X_t\}$, the frame difference sequence is computed as:

$$\begin{aligned} D_k &= X_k - X_{k-1}, \\ &k = t - T + 2, \ldots, t \end{aligned} \tag{11}$$

For the first frame in the sequence, the difference is set to zero to maintain dimensional consistency. Each time step is then represented by a two-channel input consisting of the raw frame and its corresponding frame difference:

$$I_k = [X_k,\ D_k] \tag{12}$$

This combined representation is passed through the same CNN encoder, producing motion-aware feature maps. The ConvLSTM processes the encoded sequence to model temporal dependencies, and the decoder predicts the next frame as in the original architecture.

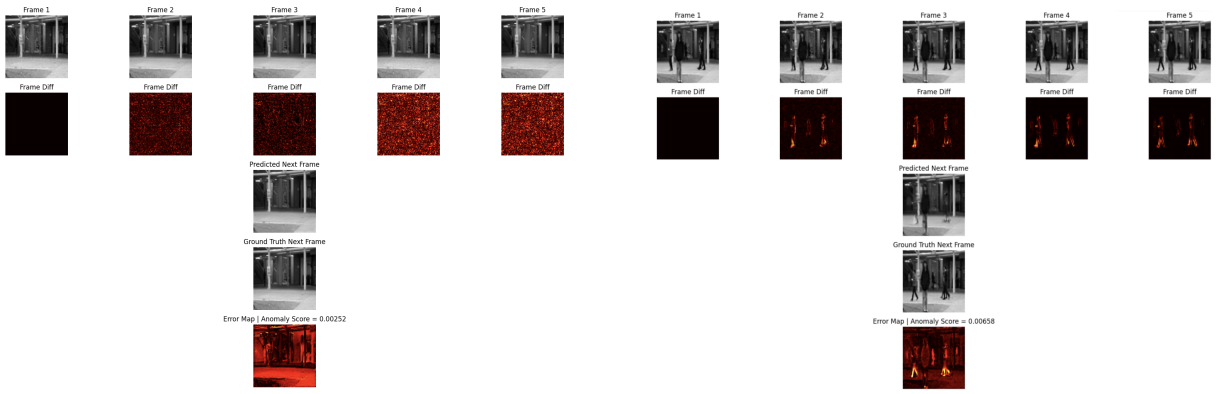
4.3.1 Training Objective and Optimization

The frame-difference enhanced CNN + ConvLSTM model is trained using the same next-frame prediction objective described previously. Given a sequence of T input frames and their corresponding frame differences, the model predicts the next grayscale frame $\hat{X}_{t+1}$.

The model is optimized using the Adam optimizer with a learning rate of 10^{-4}. The ConvLSTM hidden state size is set to 96, and training is performed with a batch size of 8. All other architectural and optimization settings remain unchanged from the previous model.

4.3.2 Anomaly Scoring

At inference time, anomaly scores are computed using the same reconstruction-based strategy. For each time step, the prediction error between the predicted frame and the actual next frame is used as the frame-level anomaly score.



(a) Normal sequence (Anomaly score = 0.0025)

(b) Anomalous sequence (Anomaly score = 0.065)

Figure 9: Qualitative comparison between a normal and an anomalous test sequence. Each example shows five consecutive input frames, corresponding to the sequence length used by the CNN–ConvLSTM model, followed by frame diff visualization,and the prediction error visualization. Brighter regions indicate higher deviation between the predicted and ground-truth next frame. The normal sequence exhibits less errors, resulting in a low anomaly score (0.02), whereas the anomalous sequence shows bright regions around motion, leading to a higher anomaly score (0.06).

4.3.3 Result

When the ConvLSTM hidden state size was set to 64, the CNN+ConvLSTM model achieved a mean average precision (mAP) of **0.54**. Increasing the hidden state size to 96 improved the performance, resulting in a higher mAP of **0.62** under the same training and inference settings. This indicates that a larger hidden state enables the ConvLSTM to retain richer temporal information across frames, leading to better modeling of motion-based anomalies.

5 Post-processing Strategies for Improving mAP

Although the CNN–ConvLSTM model generates frame-level anomaly scores using prediction error, the raw outputs are noisy and inconsistent across different videos. When used directly, these scores produce unstable rankings and reduce mean average precision. Based on this observation, I introduced several post-processing steps to stabilize the scores and improve their discriminative power during evaluation.

5.1 Fusion with Baseline CNN Autoencoder

The CNN–ConvLSTM model is primarily sensitive to temporal irregularities, whereas a frame-based convolutional autoencoder captures appearance-level anomalies. To leverage the complementary strengths of both models, a weighted fusion of their anomaly scores was performed. Let S_{ConvLSTM} denote the anomaly score from the CNN–ConvLSTM model and S_{CAE} the score from the baseline convolutional autoencoder. The fused score is computed as:

$$S = \alpha\, S_{\text{ConvLSTM}} + (1 - \alpha)\, S_{\text{CAE}}$$

I varied the fusion weight α.My Initial experiments showed steady improvement as α increased from 0.8 to 0.95. At $\alpha = 0.95$, the best performance was achieved. Values lower than this underweighted temporal information, while values higher than 0.95 caused the appearance signal to be suppressed, leading to degraded performance.

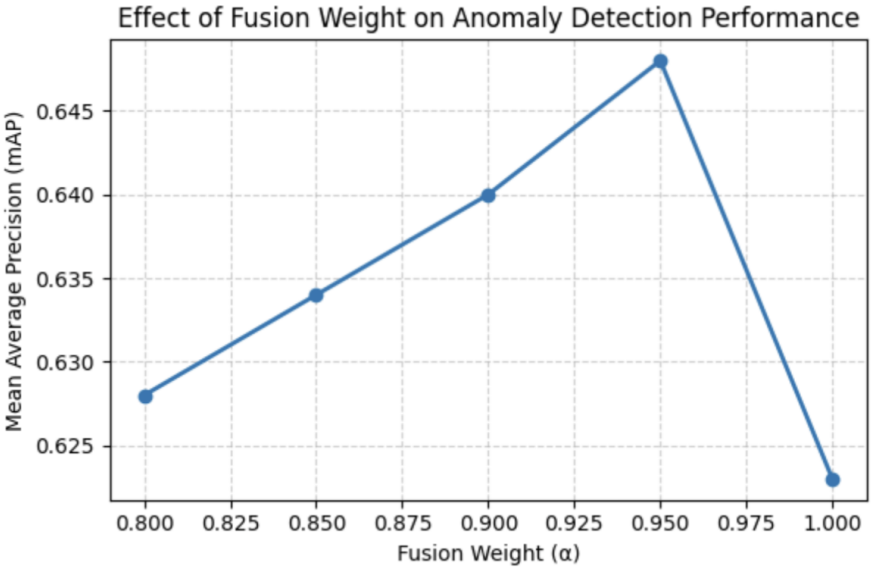


Figure 10: Mean average precision as a function of the fusion weight α

5.2 Global Score Normalization

Raw anomaly scores can vary significantly across videos due to differences in scene dynamics and reconstruction difficulty. To ensure consistent scaling across the entire test set, global min–max normalization was applied. Given the set of all anomaly scores $\{S_i\}$ across all videos, normalization is performed as:

$$\hat{S}_i = \frac{S_i - \min(S)}{\max(S) - \min(S) + \epsilon}$$

This step ensures that anomaly scores are comparable across different videos and prevents individual sequences with higher absolute errors from dominating the ranking. Global normalization consistently improved mAP by stabilizing score distributions.

5.3 Exponential Moving Average (EMA) Smoothing

Frame-level anomaly scores are inherently noisy due to prediction uncertainty and minor reconstruction artifacts. To reduce short-term fluctuations while preserving sustained anomalous behavior, Exponential Moving Average (EMA) smoothing was applied along the temporal axis. Given a sequence of scores $\{S_t\}$, the smoothed score $\tilde{S}_t$ is defined as:

$$\tilde{S}_t = \beta\, S_t + (1 - \beta)\, \tilde{S}_{t-1}$$

The smoothing factor β was tuned empirically. Lower values over-smoothed the signal and delayed anomaly peaks, while higher values failed to suppress noise. A value of $\beta = 0.6$ provided the best trade-off, leading to clearer anomaly peaks and improved ranking performance.

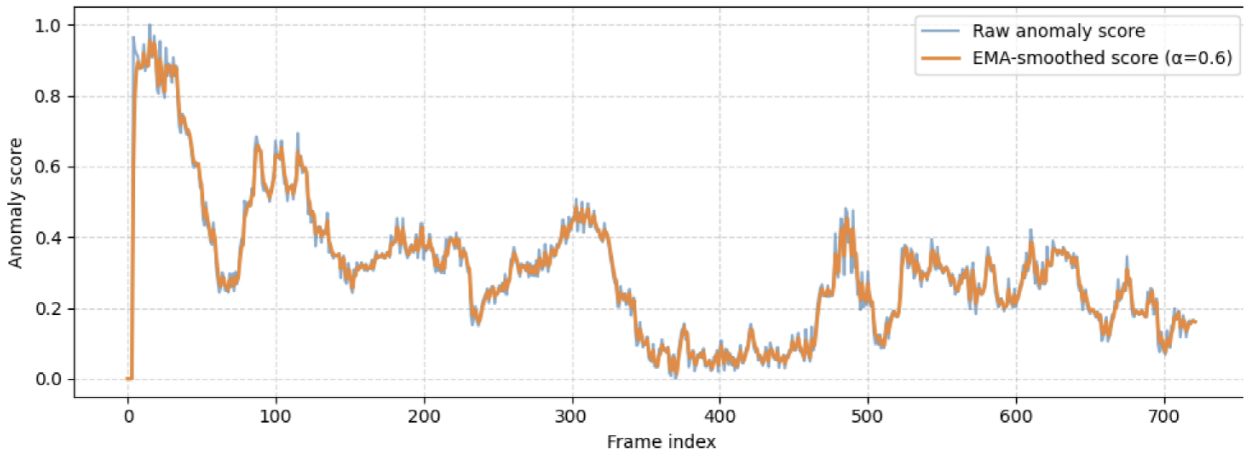


Figure 11: Comparison of raw and EMA-smoothed anomaly scores for a test video. The raw prediction error is noisy and unstable, whereas EMA smoothing reduces sudden spikes and makes anomaly trends easier to interpret across time.

5.4 Cold Start and Warm-Up Strategy

During inference, the CNN–ConvLSTM model requires a short history of frames to build a stable temporal memory. Due to the inherent design of the model, anomaly scores for the first T frames (where T is the sequence length) are automatically set to zero, since a full input sequence is not yet available. As a result, meaningful prediction-based anomaly scoring can only begin after the first T frames. However, even after this initial window, the ConvLSTM hidden and cell states are still in a early phase,as temporal memory is only partially formed. This leads to unstable predictions and noisy anomaly scores for the early part of each video. To address this, a warm-up strategy is applied during testing.Anomaly scores corresponding to the first $2T$ frames are ignored and excluded from evaluation and submission. This allows the ConvLSTM to accumulate sufficient temporal context before anomaly scoring begins,resulting in improved mean Average Precision.

5.5 Final Results

The best performance was achieved using the CNN–ConvLSTM model with frame differencing, This optimal configuration uses a ConvLSTM hidden size of 96, trained for three epochs, combined with EMA smoothing ($\beta = 0.6$) and fused with the base CNN autoencoder ($\alpha = 0.95$) and the warm-up strategy described above. Under this setting, the model achieves a mean Average Precision (mAP) of **0.66183**

Table 1: Summary of model performance across different architectures and post-processing strategies.

Model	**mAP**
CNN Autoencoder (Baseline)	0.45375
CNN + ConvLSTM	0.60236
CNN + ConvLSTM + Frame Difference (Hidden = 96)	0.62000
CNN + ConvLSTM + Frame Difference + Fusion(($\alpha = 0.95$)) + EMA ($\beta = 0.6$) + Warm-up	**0.66183**

6 Unsuccessful Experiments and Observations

In this section I explain the different experiments I tried which did not lead to performance improvements. This section briefly summarizes these experiments and provides intuition(As far as I think) for their observed behavior.

6.1 RGB Input Instead of Grayscale

I tired an RGB version of the CNN–ConvLSTM frame difference model in place of the grayscale input. However, this resulted in lower mean average precision compared to the grayscale counterpart. This behavior can be attributed to the nature of anomalies in the Avenue dataset, which are primarily motion-based rather than color-dependent. Introducing RGB channels increases input dimensionality and noise sensitivity. Moreover since the TV static data corruption is color based therefore RGB doesnt help to decrease this noise as well as its grayscale counterpart As a result, the additional color information acts as a distraction rather than a useful signal for anomaly detection. This model achieves a mean Average Precision (mAP) of **0.37**

6.2 Effect of Sequence Length Variation

I experimented with sequence length of the CNN–ConvLSTM with frame differencing study its impact on performance. Reducing the sequence length to $T = 3$ led to insufficient temporal context, causing the model to miss longer-term motion irregularities. On the other hand, increasing the sequence length to $T = 7$ resulted in degraded performance as well. Longer sequences increase temporal smoothing which reduces sensitivity to short but important anomalous events. Therefore, a sequence length of $T = 5$ provided the best balance between temporal context and anomaly sensitivity.

6.3 Incorporating Motion Volumes (`.mat` Files)

The dataset provides precomputed motion volumes stored as `.mat` files, which I tried to incorporate in as an additional input to the model. In this experiment, grayscale frames and motion volumes were stacked channel-wise and processed by the same CNN–ConvLSTM architecture. Despite careful normalization and alignment, this approach failed to improve performance. This suggests that maybe just a simple fusion of external motion representations is not sufficient and that such signals require dedicated architectural treatment rather than direct concatenation.

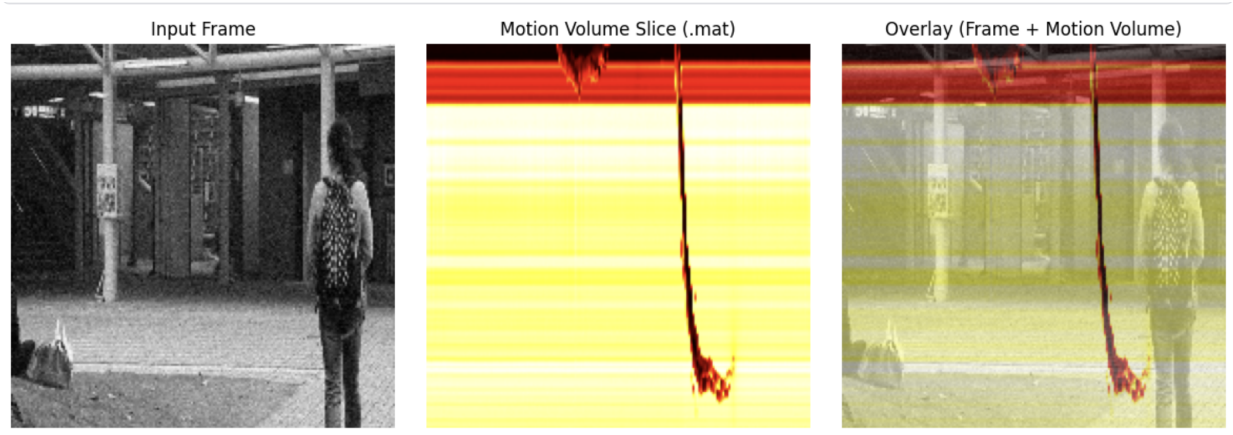


Figure 12: My intuition here is that maybe the encoder can't differentiate well between the background noise(yellow) and the actual motion path

7 Conclusion

This project provided an opportunity to work on a fully unsupervised learning problem in a realistic setting, where the data is imperfect and affected by various forms of corruption. Such conditions are common in real-world surveillance systems Working with this dataset highlighted the importance of designing models that are not only accurate, but also robust to noise, inconsistencies.

7.1 Key Challenges

A central challenge throughout this project was preventing the model from becoming too good at reconstruction. In unsupervised anomaly detection, the objective is not to perfectly reconstruct every input, but to reconstruct only normal patterns well while failing on abnormal events. Increasing model capacity too aggressively caused the network to generalize too well, resulting in anomalous events being reconstructed almost as accurately as normal ones. As reconstruction error is the primary signal used for scoring anomalies. When the model became overly dense and too good, the error gap between normal and abnormal frames reduced, leading to poor ranking performance despite low training loss. This constraint strongly influenced architectural choices. While the CNN–ConvLSTM model with frame differencing improved motion awareness, it also increased the risk of over-reconstruction. To counter this effect, dropout regularization was introduced in both the encoder and decoder. This limited the model's ability to memorize fine-grained details and encouraged it to focus on dominant normal motion patterns instead. Due to this risk of over-reconstruction, more complex architectures such as 3D CNNs and attention-based models were avoided intentionally. These models are powerful enough to reconstruct a wide range of motion patterns, including anomalies. This was quite counterintuitive to me as I often while training get the best result from the last epoch of training (usually) but in this task I got the best results using epoch 2,3,4. Overall, the need to control overfitting played a decisive role in shaping the final model design and justified the use of relatively simple architectures over more complex alternatives.

7.2 Learning Outcomes

Through this project, I developed understanding of several key concepts related to unsupervised video analysis and overall deep learning

- Understanding of convolutional autoencoders and how encoder–decoder architectures learn compact latent representations for reconstruction-based tasks.
- Practical insight into unsupervised learning, particularly how models can be trained using only normal data and evaluated based on reconstruction or prediction error.
- Experience in data visualization and exploratory analysis to understand dataset characteristics, identify corruption, and interpret model behavior.
- Familiarity with anomaly scoring mechanisms and how reconstruction or prediction errors are transformed into frame-level anomaly scores.
- Insight into evaluation metrics for anomaly detection, particularly mean average precision (mAP), and how ranking-based metrics differ from simple accuracy or loss-based evaluation.
- How to make latex documents.

References

1. CUHK Avenue Dataset. https://www.cse.cuhk.edu.hk/leojia/projects/detectabnormal/dataset.html
2. Neuronio. *An Introduction to ConvLSTM*. https://medium.com/neuronio/an-introduction-to-convlstm-55c9025563a7
3. Colah. *Understanding LSTM Networks*. https://colah.github.io/posts/2015-08-Understanding-LSTMs/
4. Michigan Deep Learning Course. *Autoencoders*. https://youtu.be/Q3HU2vEhD5Y
5. Michigan Deep Learning Course. *Recurrent Neural Networks*. https://youtu.be/igP03FXZqgo
6. Michigan Deep Learning Course. *Sequence Modeling and LSTM*. https://youtu.be/A9D6NXBJdwU
7. Michigan Deep Learning Course. *3D CNN*. https://youtu.be/S1_nCdLUQQ8?si=vHD2caN54skmb_Tx

8 Appendix

some of the results by my best model

Figure 13: Wrong direction

Figure 14: Throwing bags

Figure 15: Running person